

mini-ledger 증분 4 (1/3) — 거래 데이터 모델: TransactionType enum + Transaction 클래스

(감사 로그의 씨앗 — "무슨 일이 일어났나"를 담는 그릇을 직접 설계한 날)

한눈에

항목	내용
목표	증분 4 첫 조각 — 거래 데이터 모델(enum + Transaction 클래스)
결과	완료 ✓ — 두 파일 컴파일 0 에러 → 커밋 → 푸시
커밋	403ef81 "add TransactionType enum and Transaction model" (2 files, 25 insertions)
푸시	105c913..403ef81 main -> main
저장소	github.com/Taewoo-Won/mini-ledger (총 7개 .java)
오늘의 핵심	from/to 설계를 담 안 받고 스스로 결정 (1 vs 2 계좌 긴장 → null로 해소)
틀린 곳	14곳 (enum 2 + 필드 6 + 생성자 4 + 게터 2) — 전부 스스로 고침
새 개념	enum · 기본 타입 vs 클래스 타입 · 생성자 이름=클래스명 · 불변 필드(final) 설계 · LocalDateTime.now()
학습 방식	설계 결정은 내가, 문법 미숙은 힌트로 교정. "왜?"를 8개 캐틀어 개념 청산

코드 양은 작지만(두 파일, 25줄), 그 안에 설계 결정 3개가 박혀 있다 — from/to 모델링, null vs 빈 문자열, timestamp를 어디서 찍나. 이 셋은 베끼면 0이고 직접 풀어야 면전에서 산다. 오늘 그걸 직접 풀었다. 그리고 문법 미숙으로 14곳 틀렸지만 — "왜 그런지"를 8개 질문으로 청산한 게 진짜 수확이다.

0. 출발점 / 오늘의 목표

- 이전까지: 증분 1(Account) → 2(Bank/transfer) → 3(커스텀 예외 + try/catch) 완료. 6/26에 증분 3 복기 완성.
- 오늘 목표: 증분 4를 세 조각으로 나눈 것 중 첫 조각 = 데이터 모델.
 1. TransactionType enum (입금·출금·이체 종류)
 2. Transaction 클래스 (한 거래의 기록)
- 다음 조각(오늘 X): Bank에 List<Transaction> 이력 + 모든 거래 기록 / Main에서 이력 출력.
- 작업 도구: 폰 + VPS + nano + javac . (편집 → 컴파일 → 푸시 루프.)

PART 1 — 왜 "거래 로그"인가 (도메인 의미)

오늘 만든 건 토이 기능이 아니라 핀테크 백엔드의 심장 하나다.
결제 시스템에서 거래 로그(transaction log)는 가장 중요한 산출물이다. "언제, 어느 계좌에, 무슨 일이 일어났나"를 답할 수 있어야 한다 — 감사·규제·분쟁·이상거래 조사가 전부 이걸 읽는다.
핵심 통찰: 잔액(balance)은 *지금 상태*만 말한다. 거래 로그는 *역사 전체*를 말한다.

- 잔액 1100원 → "지금 1100원 있다"만 안다. *어떻게* 1100이 됐는지는 모른다.
- 거래 로그 → "2100에서 시작, 1000 이체 나감 → 1100" 전 과정이 남는다.

이게 로드맵 Layer 1의 감사 로그이고, Layer 0의 WAL(write-ahead log)과 사촌인 개념이다. 오늘 그 첫 그릇을 만들었다.

PART 2 — ★ 설계 결정 3가지 (오늘의 진짜 수확) ★

문법은 손에 붙으면 사라진다. 아래 3개는 직접 결정해야 했던 설계고, 이게 오늘 배움의 본체다.

2-1. from/to 모델링 — "1 vs 2 계좌" 긴장

문제: 입금·출금은 계좌가 *하나*인데, 이체는 *둘*(from·to)이다. 셋을 한 Transaction 클래스에 어떻게 담나?
판단 도구(스스로 돌린 시험):

"이 거래 하나만 1년 뒤 꺼냈을 때, 무슨 일이었는지 완전히 재구성되나?"

세 종류에 이 시험을 대봤다:

종류	from	to	재구성 되나?
입금	비용	그 계좌	"어느 계좌로 들어왔나" 남음 ✓
출금	그 계좌	비용	"어느 계좌에서 나갔나" 남음 ✓
이체	그 계좌	그 계좌	"어디서 → 어디로" 둘 다 남음 ✓

결정: from·to 둘 다 필드로 두고, 종류에 따라 한쪽을 비운다.

- 입금 = from 비고, to 채움
- 출금 = from 채우고, to 비움

- 이체 = 둘 다 채움
- 세 경우 다 시험 통과. 망설였지만 스스로이 모델에 도달했다.

(씨앗: 진짜 원장은 from/to 대신 *부호나 복식부기*로 더 깔끔히 푼다 — 그건 Layer 1이다. 지금은 from/to + 빈 쪽으로 충분하고 옳다.)

2-2. "빈다"를 코드로 — null vs 빈 문자열

문제: 출금일 때 to를 어떻게 "비우나"? 두 갈래.

선택	의미	판정
"" (빈 문자열)	"받는 계좌 id가 <i>빈 문자열</i> 이다" → id가 ""인 계좌가 있는 것처럼 읽힘. 애매	✗ 헛갈림
null	"받는 계좌가 <i>아예 없다</i> " → 값 자체가 없음. "출금이라 받는 쪽 없음"과 의미 일치	✓ 정적

결정: null . (6/26에 배운 그 null — "아무것도 없음" 신호.) 1년 뒤 to가 null인 걸 보면 "아 출금이라 받는 쪽이 없구나"가 읽힌다.

주의: Transaction 클래스 코드 자체엔 null 처리로 더 칠 게 없다. null은 값이라 String 필드에 그냥 들어간다. 실제 null을 넣는 건 나중에 **Bank가 출금 기록을 만들 때** 일이다. (다음 조각에서.)

2-3. timestamp — 밖에서 받나, 안에서 찍나

문제: 거래 시각(timestamp)을 생성자가 *매개변수로 받을지*, 아니면 *안에서 만들지*.

판단: 거래 시각은 "그 거래가 일어난 순간"이고, Transaction이 *만들어지는 순간*이 곧 그 순간이다.

결정: 생성자 안에서 LocalDateTime.now() 로 찍는다. 밖에서 안 받는다.

왜 이게 더 나은가 (감사 로그 관점): 안에서 찍으면 *호출자가 시각을 조작할 수 없다*. 거래 시각을 임의로 넣게 두면 기록을 못 믿는다 → 감사 로그의 신뢰성이 무너진다. 그래서 "받지 말고 안에서 찍기"가 옳다.

이 결정의 코드적 결과 (중요): 그래서 timestamp만 특수해진다 —

- 필드엔 있음 (저장하니까)
- 매개변수엔 없음 (밖에서 안 받으니까)
- this 대입엔 있음 (채워야 하니까, 단 now() 로)

→ 이게 PART 4에서 "필드 5 / 매개변수 4 / this 5"의 정확한 이유다.

PART 3 — TransactionType enum (내 코드 vs 정답)

3-1. 내 코드 (원문)

```
public enum TransactionType {
    deposit
    withdraw
    transfer
}
```

3-2. 정답

```
public enum TransactionType {
    DEPOSIT, WITHDRAW, TRANSFER
}
```

3-3. 줄별 차이 — 2곳

#	내가 쓴 것	정답	왜
E1	deposit (소문자)	DEPOSIT (대문자)	상수는 전부 대문자 관습. "절대 안 변하는 값"이라는 신호. 소문자도 컴파일은 되지만 관습 위반.
E2	상수 사이 구분자 없음	DEPOSIT, WITHDRAW, TRANSFER	enum 상수는 <i>실표</i> 로 잇는다. (필드처럼 세미콜론 ✗. 메서드 매개변수처럼 실표 ✓.)

3-4. enum의 정체

- enum = 고정된, 이름 붙은 값의 집합. TransactionType은 DEPOSIT·WITHDRAW·TRANSFER 셋 *외엔 존재할 수 없다*.
- 왜 String "deposit" 대신 enum? String이면 오타 "depoist" 가 조용히 통과해 런타임에 터진다. enum은 TransactionType.DEPOSIT 을 *컴파일 단계*에서 막는다. → 에러를 컴파일 타임으로 밀어내는 것. (Map 제네릭에서 배운 타입 안전성과 같은 철학.)

PART 4 — Transaction 클래스 (내 코드 vs 정답, 단계별 해부)

오늘 제일 많이 틀린 곳. 단계별로 12곳을 잡았다. 전부 *문법 미숙*이지 *논리 오류*가 아니다 — 개념(필드·생성자·게터)은 잡혔고, 손이 아직 안 붙은 것.

4-A. 시작 골격 (원문)

```
import java.time.LocalDateTime;

public class Transaction {
    private final Localdatetime
    private final TransactionType
    private final long

    public Transaction(TransactionType, long amount) {
    }

    public LocalDateTime getTimestamp() {
        return this.timestamp;
    }
}
```

문제 한눈에: 필드에 *이름이 없음*, Localdatetime 오타, 세미콜론 없음, 생성자 몸통 비어 있음, 계좌 필드(from/to) 통째로 빠짐, 매개변수 불완전.

4-B. 필드 선언 — 6곳 해부

중간 시도 (원문)

```
private final TransactionType(type);
private final from(String);
private final to(String);
private final amount(long);
private final timestamp(LocalDateTime);
```

정답

```
private final TransactionType type;
private final String from;
private final String to;
private final long amount;
private final LocalDateTime timestamp;
```

줄별 차이

#	내가 쓴 것	정답	규칙
E3	필드에 <i>이름 없음</i> (...TransactionType)	이름 붙임 (...type;)	필드 = 접근제어 final 타입 이름 ; (1일차 private final String id; 그 패턴)
E4	Localdatetime	LocalDateTime	철자 — D 대문자 . (IntelliJ였으면 빨간 줄로 즉시 잡혔을 것.)
E5	세미콜론 없음	; 추가	문장 끝 세미콜론
E6	TransactionType(type); / from(String);	TransactionType type; / String from;	타입 먼저, 이름 나중, 괄호 없음. 줄마다 순서가 뒤죽박죽이었음 (어떤 줄은 이름이 앞, 어떤 줄은 타입이 앞).
E7	LocalDateTime Timestamp; (대문자 T)	LocalDateTime timestamp;	필드명 소문자 시작. 생성자의 this.timestamp 와 대소문자까지 일치해야 함. 안 맞으면 "그런 필드 없음" 에러.
E8	계좌 필드(from/to) 없음	from·to 추가	설계 결정(PART 2-1). 감사 기록엔 "어느 계좌인가"가 필수. 양으로 작아도 개념적으로 제일 무거운 수정.

E6 핵심 교훈: "괄호만 빼면 되나?" → 아니다. 줄마다 타입/이름 순서가 달라서, *괄호 빼고 + 순서도 통일*해야 했다. 규칙은 하나: **타입이 항상 이름 앞**.

4-C. 생성자 — 4곳 해부

중간 시도들 (원문, 시간순)

```
// 시도 ①: 생성자 이름이 틀림
public timestamp(type, from, to, amount) {

// 시도 ②: 이름은 고쳤으나 매개변수에 타입 없음
public Transaction(type, from, to, amount) {

// 시도 ③: 타입 붙였으나 두 개가 틀림
public Transaction(Transaction type, String from, String to, Long amount) {
```

정답

```
public Transaction(TransactionType type, String from, String to, long amount) {
    this.type = type;
    this.from = from;
    this.to = to;
    this.amount = amount;
    this.timestamp = LocalDateTime.now();
}
```

```
}
```

줄별 차이

#	내가 쓴 것	정답	규칙
E9	<code>public timestamp(...)</code>	<code>public Transaction(...)</code>	생성자 이름 = 클래스 이름 (대소문자까지). <code>timestamp</code> 는 필드 이름이지 클래스 이름이 아님. (26일 규칙 재등장.)
E10	(type, from, to, amount) 타입 없음	(TransactionType type, ...)	매개변수도 타입 이름. 생성자가 "필 받는지" 선언하는 자리라 타입 필수.
E11	<code>Transaction type</code>	<code>TransactionType type</code>	Transaction (클래스) ≠ TransactionType (enum) . 헷갈리는 이름이지만, <i>필드 선언과 같은 타입을 쓰면 안 됨</i> .
E12	<code>Long amount</code>	<code>long amount</code>	기본 타입은 소문자. <code>long</code> (기본 정수) vs <code>Long</code> (객체로 감싼 것)은 다른 것. 돈은 <code>long</code> . (PART 6-2 참조.)

`this.~ 5줄 + now()` 는 처음부터 완벽했다. 받은 값을 필드에 넣는 것, `timestamp`만 `LocalDateTime.now()` 로 찍는 것 — 설계 의도대로 정확히 짰다. 거긴 손 안 댔.

4-D. 게터 — 2곳 해부

정답

```
public TransactionType getType() { return this.type; }
public String getFrom() { return this.from; }
public String getTo() { return this.to; }
public long getAmount() { return this.amount; }
public LocalDateTime getTimestamp() { return this.timestamp; }
```

#	틀린 것	정답	규칙
E13	<code>getTimestamp</code> 만 있음	게터 5개로 확장	Main에서 출력하려면 <i>각 필드를 꺼낼 통로</i> 가 필요. 게터의 반환 타입 = 그 필드의 타입.
E14	게터를 생성자 안에 넣으려 함	생성자 밖, 클래스 안(형제)	게터와 생성자는 <i>형제</i> 지 부모자식 아님. 생성자 } 로 닫은 <i>다음</i> 줄부터 게터. (Account에서 <code>deposit</code> 이 생성자 밖에 나란히 있던 것과 같음.)

PART 5 — ★ 풀코드 비교: 최초 골격 vs 최종 완성 ★

5-1. Transaction.java — 최초(깨짐) vs 최종(컴파일 통과)

최초 골격 (다수 오류)	최종 완성 (컴파일 0 에러)
<pre>import java.time.LocalDateTime; public class Transaction { private final LocaldateTime private final TransactionType private final long (이름·세미콜론·계좌필드 없음) public Transaction(TransactionType, long amount){ } ← 몸통 비어 있음 public LocalDateTime getTimestamp() { return this.timestamp; } }</pre>	<pre>import java.time.LocalDateTime; public class Transaction { private final TransactionType type; private final String from; private final String to; private final long amount; private final LocalDateTime timestamp; public Transaction(TransactionType type, String from, String to, long amount){ this.type = type; this.from = from; this.to = to; this.amount = amount; this.timestamp = LocalDateTime.now(); } public TransactionType getType() { return this.type; } public String getFrom() { return this.from; } public String getTo() { return this.to; } public long getAmount() { return this.amount; } public LocalDateTime getTimestamp() { return this.timestamp; } }</pre>

5-2. 최종 Transaction.java (전체, 그대로)

```
import java.time.LocalDateTime;

public class Transaction {
```

```

private final TransactionType type;
private final String from;
private final String to;
private final long amount;
private final LocalDateTime timestamp;

public Transaction(TransactionType type, String from, String to, long amount) {
    this.type = type;
    this.from = from;
    this.to = to;
    this.amount = amount;
    this.timestamp = LocalDateTime.now();
}

public TransactionType getType() { return this.type; }
public String getFrom() { return this.from; }
public String getTo() { return this.to; }
public long getAmount() { return this.amount; }
public LocalDateTime getTimestamp() { return this.timestamp; }
}

```

5-3. 최종 TransactionType.java (전체)

```

public enum TransactionType {
    DEPOSIT, WITHDRAW, TRANSFER
}

```

PART 6 — ★ "왜?" 8문답 (개념 청산) ★

빈칸 채우기로 넘기지 않고, *왜 그런지*를 끝까지 캐물었다. 이게 면접에서 꼬리질문으로 들어오는 지점이다.

1. 생성자 이름이 왜 클래스 이름과 같나? 같으면 중복 아닌가?

중복 아니다 — *종류가 다르다*. `public class Transaction` 은 **클래스 선언**, `public Transaction(...)` 은 **생성자**. 자바 문법이 "생성자 이름 = 클래스 이름"으로 정해놔고, 그게 자바가 "이 메서드는 *생성자구나*"를 알아보는 *방법*이다. 위아래가 같은 게 **맞는** 거다.

2. 왜 class 가 있으면 3명이, 없으면 2명이?

`class` 키워드 유무가 "클래스 선언이나 메서드냐"를 가른다.

- `public class Transaction` (3명이) = "클래스를 *정의*한다"
- `public Transaction(...)` (2명이) = 생성자 (메서드라 `class` 없음)
- → **딩이 개수로 외우면 안 된다**. `class` 유무를 보라.

3. 그럼 2명이면 무조건 생성자인가?

아니다. 진짜 판별 기준은 **반환 타입 유무**:

- 이름 앞에 타입 있음(`void` / `long` / `Account` ...) → **메서드** (예: `public void run()` 은 3명이 메서드)
- 이름 앞에 타입 없음 + 이름이 클래스명 → **생성자**
- 딩이 개수는 우연일 뿐. *타입이 붙었냐*를 보라.

4. void 들어가면 메서드인가?

맞다, 근데 더 정확히는 **반환 타입이 하나라도 있으면 메서드**. `void` (돌려줄 것 없음) · `long` · `String` · `Account` 다 메서드 신호. `void` 는 그중 "반환값 없는" 경우.

5. 왜 long 만 소문자고 String · TransactionType 은 대문자?

타입이 두 종류라서.

- **기본 타입(primitive)**: `long`, `int`, `double`, `boolean` — 자바에 원래 박힌 기본 값, 전부 **소문자**.
- **클래스 타입**: `String`, `TransactionType`, `LocalDateTime` — 누군가 클래스로 만든 것, 관습상 **대문자 시작**.
- `Long` (대문자)도 있지만 그건 `long`을 *객체로 감싼 클래스*라 별개. 돈은 기본 타입 `long`.

6. 필드 5개 / 매개변수 4개 / this 5개 — 기준이 뭔가?

세 자리는 *역할이 다르다*.

- **필드 5** = 객체가 가질 데이터 전부.
- **매개변수 4** = 밖에서 *받을* 것만. `timestamp`는 안 받으니까 빠져서 4개.
- **this 5** = *채울* 것 전부. 4줄은 받은 값으로, 마지막 1줄(`timestamp`)은 `now()` 로.

7. 왜 timestamp만 특수한가?

나머지 넷은 "밖에서 받아서 → 필드에"(매개변수 0)인데, `timestamp`만 "안에서 만들어서 → 필드에"(매개변수 X, `now()`). 이게 "시각은 밖에서 받지 말고 안에서 찍자"는 설계 결정(PART 2-3)의 코드적 결과다.

8. 게터 중괄호 안 띄어쓰기({ return ...; })는 왜? 붙이면 안 되나?

붙여도 100% 작동한다 — 자바는 공백을 무시한다(들여쓰기 안 맞아도 에러 아니던 것과 같음). 띄우는 건 **순수 가독성, 기능 0**. 자바 공식 관습(구글/오라클 스타일)이라 거의 다 띄운다. 평가관이 읽을 코드니 관습 따른다.

제일 중요한 건 1·3·7. 생성자의 정체(1), 메서드/생성자 판별법(3), 설계 결정이 코드 구조로 드러나는 것(7) — 베끼면 안 보이고, 직접 치며 물으니 보였다.

7. 산출물

- 파일 (`~/mini-ledger/` , 총 7개 .java):

- `TransactionType.java` — 신규 (거래 종류 enum)
- `Transaction.java` — 신규 (한 거래의 기록: type·from·to·amount·timestamp)
- `Account / Bank / AccountNotFoundException / InsufficientBalanceException / Main` — 변경 없음
- 컴파일·검증:
`bash cd ~/mini-ledger javac *.java # 에러 0 (.class 7개 생성 확인) git status # Transaction.java, TransactionType.java untracked / .class는 안 보임(정상)`
- 커밋·푸시:
`[main 403ef81] add TransactionType enum and Transaction model 2 files changed, 25 insertions(+) create mode 100644 Transaction.java create mode 100644 TransactionType.java 105c913..403ef81 main -> main`
- mini-ledger 코드 커밋 계보:
`9e671d6 (Account) → e02b204 (Bank createAccount) → 9cc49e6 (transfer+Main) → 532825f (커스텀 예외+try/catch) → 403ef81 (enum+Transaction 모델)`

8. 다음 작업 (증분 4, 2/3)

Bank에 `List<Transaction>` 이력 + 모든 거래 기록.

- 새 도구: `List` (제네릭, Map의 사촌) + `.add()`.
- 진짜 설계 문제 (다음 세션의 핵심): `Account.deposit` 은 Bank를 *모른다*. 그럼 이력이 Bank에 있을 때 *어디서* 기록하나?
- 답의 방향: **Bank를 유일한 출입구로 만든다**. 모든 돈 이동이 Bank를 통과 → 전부 기록됨 = 감사 로그. (Bank에 `deposit/withdraw` 메서드를 뒤서 funnel로.)
- 그 다음(3/3): Main에서 입금·출금·이체 굴리고 이력이 쌓이나 눈으로 확인.

9. 개념 사전

- **enum**: 고정된, 이름 붙은 값의 집합. 상수는 대문자, 심표로 구분. String보다 타입 안전(오타를 컴파일 타임에 차단).
- **불변 필드 (final)**: 한 번 정해지면 재할당 불가. 거래 기록은 append-only라 모든 필드 `final`. (1일차 `Account.id` 의 그 키워드.)
- **기본 타입 vs 클래스 타입**: `long / int` (기본, 소문자) ↔ `String / LocalDateTime` (클래스, 대문자). `long ≠ Long` (후자는 객체).
- **생성자 식별**: 이름=클래스명 + 반환 타입 없음. `class` 키워드 없음.
- **메서드 식별**: 반환 타입 있음 (`void` 포함). 게터도 메서드(반환 타입 = 필드 타입).
- **필드/매개변수/this 구분**: 가질 것(필드) / 받을 것(매개변수) / 채울 것(this). 안 받고 안에서 만드는 값(timestamp)은 매개변수에서 빠짐.
- `LocalDateTime / .now()`: 날짜+시각 타입 / 지금 이 순간을 찍는 명령. 생성자 안에서 찍으면 호출자가 시각 조작 불가(감사 신뢰성).
- **null vs ""**: `null=""`아예 없음(출금의 to에 적합), `""=""`빈 문자열 값(id가 ""인 것처럼 오해). 비움은 `null`로.
- **감사 로그 / append-only**: 한 번 쓰이면 안 바뀌는 거래 기록. "무슨 일이 일어났나"의 진실 원본. Layer 1 / WAL의 씨앗.

10. 비유 모음

- **거래 로그 = 통장 거래내역**: 잔액은 "지금 얼마"만, 거래내역은 "어떻게 그 잔액이 됐나" 전부. 한 줄도 못 고친다(감사).
- **from/to + null = 일방통행 vs 양방향**: 입금/출금은 한쪽만 있는 일방(반대쪽 null), 이체만 양쪽 다 있음.
- **enum = 정해진 보기 객관식**: "주관식(String)"이면 오타로 아무거나 쓰지만, "객관식(enum)"은 DEPOSIT·WITHDRAW·TRANSFER 셋 중 하나만 고를 수 있다.
- **생성자 = 봉어빵 굽는 공정**: `class Transaction (봉어빵 틀) / Transaction(...)` (그 틀로 굽는 공정). 공정 이름을 틀 이름과 같게 지어야 "이 틀의 공정"인 걸 자바가 안다.
- **기본 타입 vs 클래스 타입 = 기성 부품 vs 조립품**: `long` 은 자바에 원래 있는 기성 부품(소문자), `String · LocalDateTime` 은 누가 조립해 만든 부품(대문자).

11. 회고

오늘 14곳 틀렸다. 근데 패턴이 보인다: **전부 문법 미숙이지 논리 오류가 아니다**. 타입을 이름 앞에 놓기, 생성자 이름을 클래스명과 맞추기, 기본 타입 소문자, 매개변수에 타입 붙이기 — 손에 붙으면 사라질 것들이다. 진짜 개념 오류는 0이었다. 설계(from/to, null, now())는 처음부터 옳게 갔다.

그리고 — **설계 결정 3개를 답 안 받고 직접 내렸다**. from/to를 둘 다 두고 한쪽을 null로 비우는 모델, 시각을 안에서 `now()` 로 찍는 결정. 이걸 베꼈으면 못 가졌을 판단이고, "1년 뒤 재구성되나?"라는 시험을 스스로 돌려 도달했다. 망설였지만("많이 틀렸겠다") 실제론 핵심을 맞췄다 — 자기 의심과 실제 실력은 다르다.

문법으로 막힐 때마다 "왜?"를 묻은 게 8개. 생성자가 왜 클래스명인지, long은 왜 소문자인지, timestamp는 왜 특수한지 — 이걸 청산한 게 오늘의 본체다. **모르는 걸 모른다고 짚어 묻은 것 자체가 이해의 시작이다**.

다음은 증분 4 둘째 조각 — `List<Transaction>` + Bank를 출입구로. `Account.deposit` 이 Bank를 모르는데 이력은 Bank에 있는, 그 설계 문제가 다음 산이다.

한 줄: 25줄짜리 두 파일이지만, 그 안에 **직접 내린 설계 결정 3개**와 **직접 캐물은 "왜" 8개**가 들어 있다. 양이 아니라 그게 오늘이다.